Retrofitting Pretrained Transformers with Progressive Retrieval Attention: Thin Hugging Face Adapters and a Qwen Correctness Study

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 11, 2026

Abstract

Progressive Retrieval Attention (PRA) selects sparse, layer-native key/value (K/V) memory from references or displaced prompt history. Controlled models establish the mechanism but do not show that it can preserve a real pretrained transformer's attention semantics. We begin a pretrained integration study with a shared PRA execution core and thin Hugging Face attention adapters. The first adapter wraps one upper layer of the frozen 596M-parameter `Qwen/Qwen3-0.6B` checkpoint. When memory is disabled, the wrapper delegates to the original module and obtains bit-exact logits, hidden states, greedy generation, and all 28 generation-cache shapes. When memory is enabled, it reuses Qwen's pretrained projections, Q/K normalization, rotary embedding, grouped-query layout, causal mask, cache update, eager attention function, and output projection. Stored memory retains eight native K/V heads rather than expanding to 16 query heads. A CUDA smoke run materializes reference K/V from CPU and executes a 408-token logical prompt under a 256-token native-operation bound. A subsequent evidence-ranking study separates routing from materialization: post-RoPE key means, pre-RoPE key means, and attention-input hidden-state means all retain the same post-RoPE detail K/V. On 16 matched HotpotQA/QASPER examples with 32-token chunks, post-RoPE routing has a strong late- position correlation ($r = 0.652$) and recall@3 of 0.125. Pre-RoPE routing removes that correlation ($r = 0.009$), while hidden-state routing gives the best sparse recall@3, 0.438. Two confirmation subsets totaling 64 evaluations give hidden-state recall@3/8/16 of 0.391/0.578/0.797, but QASPER recall@3 is only 0.156 and recall@16 selects 23.8% of chunks. Splitting each 32-token parent into 2, 4, or 8 contiguous mean gists does not improve sparse ranking: recall@3 falls to 0.313/0.266/0.281 and MRR falls in every condition. A token-resolution maximum over 32 hidden states also fails as a general remedy: recall@3 changes from 0.438 to 0.375 while routing-index overhead grows from 1.57% to 50% of detail-K/V bytes. A final centered-RoPE control pools pre-RoPE keys and then rotates once at the exact span center. It improves recall@3 over a post-RoPE mean from 0.125 to 0.188, but remains below pre-RoPE (0.313) and hidden-state (0.438) routing. With eight centered subgists, correlation with native token-QK maximum ordering rises to 0.874 while recall@3 reaches only 0.250. Thus semantic selection and positional K/V transport should be separated, but parameter-free Qwen routing does not pass the predeclared gate for Llama integration. A held-out query study then rejects recent-state and question-span pooling: the last state obtains recall@3/MRR of 0.469/0.413, versus 0.250/0.279 for the strongest question-decay candidate. Evidence-supervised projections improve retrieval. Validation across capacity and objective controls selects an asymmetric 128-dimensional margin router with 262,144 parameters (0.044% of Qwen). Across five seeds it raises held-out recall@3 from 0.469 to 0.631 ± 0.069 (95% half-width), with HotpotQA/QASPER recall@3 of 0.588/0.675 and recall@8 of 0.806. Width 256, sampled negatives, and one hard-negative refresh do not improve this selection. Cross-domain recall@3 remains 0.213 in both

directions, the 0.70 promotion gate is not met, and an end-to-end check changes retrieval without changing answer F1.

1 Introduction

Pretrained decoder families differ in projection layout, rotary implementation, grouped-query attention (GQA), masks, sliding windows, and generation-cache conventions. Replacing a complete model class to add memory duplicates rapidly changing library code and makes parity difficult to audit. The alternative studied here is narrower:

$$\text{normal pretrained model} \rightarrow \text{wrap selected attention modules} \rightarrow \text{shared PRA core.}$$

The family adapter owns only native attention semantics. The shared core owns references, chunk gists, exact routing, the materialization budget, selected K/V transfer, and metrics. This boundary follows the broader transformer interface of queries attending to keys and values [9], while preserving Qwen's pretrained RoPE and GQA choices rather than approximating them in a parallel implementation [8, 1, 7].

This paper begins with Qwen because a sub-billion-parameter checkpoint makes exact tests practical on constrained hardware while still exercising modern RoPE and GQA. It does not yet claim cross-family portability or useful long-context QA. The contribution is a tested first adapter and an evidence hierarchy that prevents working transport from being mistaken for working retrieval.

2 PRA Recap and Prior Contracts

For layer ℓ, PRA stores reference chunks as native $K_c^{(\ell)}, V_c^{(\ell)} \in \mathbb{R}^{H_{kv} \times T_c \times d_h}$ and one or more compact gists $g_c^{(\ell)}$. A query representation $r^{(\ell)}$ scores the gists, and a global budgeter selects whole chunks. Materialized memory precedes the direct K/V sequence under one softmax:

$$\mathrm{Attn}(Q, [K_M; K_D], [V_M; V_D]) = \mathrm{softmax}\left(\frac{Q[K_M; K_D]^\top}{\sqrt{d_h}} + M\right)[V_M; V_D]. \tag{1}$$

This is not a separately normalized cross-attention residual. The native output projection is applied once to the combined result.

Paper 1 established controlled native-K/V routing and materialization. Paper 1.5 separated logical source position from contextual fragmentation. Continuous sources use $p_i = p_{\text{logical start}} + i$, and ordinary post-RoPE keys remain the canonical cache state. The present adapter preserves that contract by capturing each selected HF layer's actual post-RoPE keys and by assigning source-relative positions across bounded encoding blocks.

3 Adapter Architecture

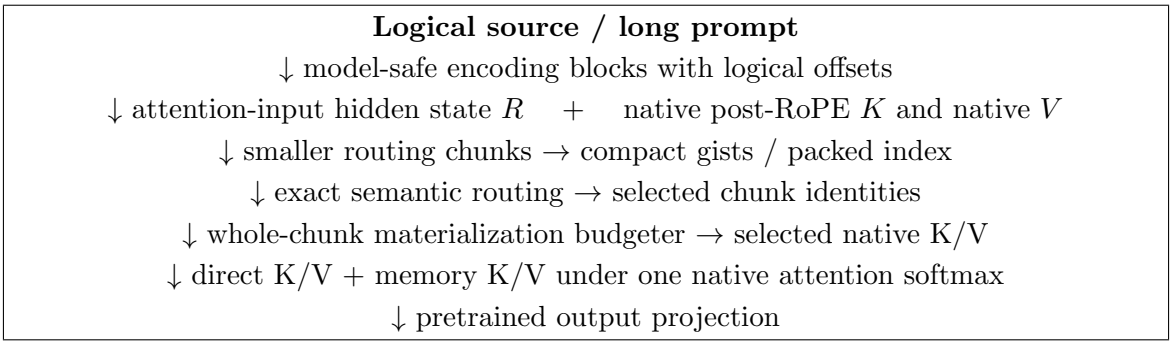

Figure 1: Canonical HF path. Routing uses compact semantic state, whereas attention receives the host model's selected native positional K/V payload. Model-family projection, RoPE, GQA, mask, cache, and output semantics remain in the thin adapter.

3.1 Routing State Is Not Attention State

PRA represents a cached chunk as

$$\mathcal{M}_c = (R_c, K_c, V_c), \tag{2}$$

where R_c is a compact routing representation and (K_c, V_c) is the payload used after selection. The objects need not inhabit the same feature space. The canonical HF configuration in this paper sets R_c to one or more gists of the hidden state entering the selected attention block, while K_c is that block's native post-RoPE key tensor and V_c its native value tensor. Token-level hidden states are transient during publication; only compact gists and detail K/V persist.

This split follows the specialization chain

$$h_{\text{in}} \longrightarrow W_K h_{\text{in}} \longrightarrow \text{RoPE}(W_K h_{\text{in}}). \tag{3}$$

The first representation retains broad contextual information, the second is an attention-address representation, and the third conditions that address on token position. Native token attention needs the third. Chunk retrieval may prefer the first. The routing query therefore uses the final attention-input hidden state and is compared only with hidden-state gists. It is never compared directly with Q_{post}. This is an empirical default for the tested Qwen checkpoint, not a claim that hidden states are universally optimal routers.

3.2 Shared execution core

The controlled model's former monolithic attention forward pass was divided into reusable operations. `prepare_pra_query` selects the newest valid query and maps GQA query groups into the native K/V routing width. `route_memory` calls the exact packed-index router. `budget_selected_memory` enforces

$$L_{\text{direct}} + L_{\text{materialized}} + L_{\text{reserve}} \leq L_{\text{native,max}}. \tag{4}$$

`materialize_selected_kv` removes duplicate overlap, preserves row isolation, and moves only retained detail K/V. `combine_local_and_memory_kv` pads variable rows and prepends an additive all-visible memory mask to the family's existing local causal mask. The controlled model and HF adapters now call this same core.

3.3 Thin HF contract

The abstract adapter exposes six family operations: project Q/K/V, apply native position encoding, normalize tensor layout, combine the native mask, invoke PRA attention, and project the output. Disabled memory takes an even narrower path: call the original attention object unchanged. This design preserves pretrained parameters and avoids checkpoint conversion.

The Qwen adapter has 272 physical lines (248 nonblank, non-comment lines). It introduces no learned parameter. Its family-specific branches distinguish Qwen2-style projections from Qwen3's Q/K normalization; routing, budgeting, materialization, residency, and metrics remain outside the file. This is a portability measurement, not a code-minimization target.

4 Qwen Native Semantics

4.1 Projection and RoPE

The adapter calls the wrapped module's `q_proj`, `k_proj`, and `v_proj`. Qwen3's existing `q_norm` and `k_norm` are applied before the installed Transformers `apply_rotary_pos_emb` function. Reference capture saves post-RoPE K and position-independent V. No independent RoPE implementation is introduced.

4.2 Grouped-query attention

The tested checkpoint has $H_q = 16$, $H_{kv} = 8$, and $d_h = 128$. Cache objects retain $[B, 8, T, 128]$ K/V. Key-space routing baselines average each pair of query heads sharing one K/V head, producing a vector in the same 8×128 space as a flattened key gist. The canonical hidden-state router instead uses one d_{model} vector and does not alter payload head layout. K/V expansion occurs inside Qwen's eager attention function, never in persistent PRA storage. A synthetic replay test compares this path with explicit K/V head repetition and matches within 10^{-6}.

4.3 HF cache and masks

Prefill and single-token decode both use the wrapped module's HF `Cache.update` call. PRA memory is prepended after that update, so direct generation history appears once. A decode test with four prompt tokens and two generated tokens observes five direct cache positions at the last invocation, plus four separate memory positions. The original causal mask remains on local/cache positions; selected historical memory receives a visible prefix mask. Memory- disabled generation delegates entirely to the original module.

5 Experimental Setup

Table 1 records the first checkpoint exactly. Tests use synthetic Qwen3 configs offline. The pretrained run uses Python 3.10.11, PyTorch 2.12.1+CUDA 12.6, Transformers 4.55.4, eager attention, FP16, and an NVIDIA GeForce GTX 950M. The frozen model is not fine-tuned.

Table 1: Pinned Qwen checkpoint and PRA placement.

Checkpoint	`Qwen/Qwen3-0.6B`
Revision	`c1899de289a04d12100db370d81485cdf75e47ca`
Parameters / layers	596,049,920 / 28
Query heads / K/V heads / head width	16 / 8 / 128
RoPE base / advertised positions	10^6 / 40,960
PRA placement	layer 27 only
Initial gist / routing policy	mean, one gist/chunk, exact top-k
Native limit / direct / selected-memory cap	256 / 128 / 64 tokens
K/V residency	CPU full cache, selected transfer

Every run writes JSON and scalar CSV with the Git SHA, model revision, software versions, device, dtype, layer selection, limits, routing settings, timings, and metrics. The exact executed commit is retained in each artifact rather than inferred from the manuscript build.

6 Adapter Parity

PRA-disabled parity is the hard gate. The baseline is evaluated first; the same in-memory model is then wrapped, leaving every parameter object in place. Table 2 reports exact results for the five-token prompt "The capital of Portugal is" and four-token greedy decode.

Table 2: Original HF model versus memory-disabled PRA wrapper.

Metric	Result
Maximum / mean absolute logit difference	0 / 0
Maximum hidden-state difference	0
Greedy token agreement	100%
Greedy sequences exactly equal	yes
Generation-cache shapes equal	all 28 layers
Finite adapted logits	yes

Single-run prefill times were 0.447 s before wrapping and 0.136 s after wrapping. This order is confounded by first-call warm-up and is not a speed comparison. Exact outputs, rather than latency, are the parity evidence.

7 Native-KV Reference Smoke

An explicit 22-token reference about Lisbon is encoded through the frozen model. The selected layer captures $[1, 8, 22, 128]$ K/V, stores it on CPU, creates a mean gist, and transfers all 22 tokens because the source fits one routing chunk. The physical transfer is 90,112 bytes, exactly $2 \times 22 \times 8 \times 128 \times 2$ bytes for FP16 K and V. The run reports 1.6 ms routing, 0.40 ms materialization orchestration, 0.15 ms selected transfer, and 0.11 ms combined attention. These synchronized phase values are instrumentation checks on one machine, not throughput claims. Cold reference construction took 0.144 s and the warm query took 0.191 s.

The generated continuation contains "Lisbon", but the unmodified model also knows this fact. Consequently, this observation establishes functional selected-K/V transport and generation, not a

causal retrieval benefit. The exact native-replay claim is instead restricted to the synthetic tensor test, where GQA memory plus local K/V equals explicit dense composition.

8 Implicit Head and Bounded Long Prompt

A repeated prompt tokenizes to 408 logical tokens. The direct cap retains the final 128 tokens; the first 280 become `#_head`. That head is encoded in blocks of at most 64 tokens, with source-relative positions. The direct tail receives positions 280 through 407. Every encoding operation and the final selected-memory operation stays under the 256-token configured native limit; the largest observed source-encoding call is 64 tokens and violations are zero. The final finite-logit query took 0.258 s. This proves bounded execution and offset propagation for the frozen Qwen adapter. It does not show that the router selected the information needed by an arbitrary continuation.

9 Unrestricted QA Smoke

We next run one fixed HotpotQA example [10] and one QASPER example [3]. Four conditions use the same frozen checkpoint: question-only truncation, dense text left-truncated to 256 tokens, oracle evidence text-RAG, and PRA with the question direct and the complete source in native-K/V memory. Oracle text-RAG is an upper control because it receives annotated evidence. This two-example matrix diagnoses plumbing; it cannot estimate dataset accuracy.

Table 3: Unrestricted-QA smoke outcomes. Standard normalized token F1 is shown.

Dataset	Condition	F1	Annotated evidence selected
HotpotQA	truncation; dense; oracle text-RAG; PRA	0; 0; 0; 0	PRA: 0%
QASPER	truncation; dense; oracle text-RAG; PRA	0; 0.182; 0; 0.182	PRA: 0%

HotpotQA's expected answer is "yes"; every condition begins with "No". QASPER's expected answer is "no"; dense and PRA generations begin with "No", while truncation begins with "Yes". However, PRA routes three tail chunks, none overlapping the annotated evidence, so the QASPER output cannot be credited to retrieved evidence. Qwen also fails the oracle text-RAG control on both examples. The proper conclusion is negative: this frozen top-layer, one-gist-per-chunk configuration has not demonstrated content-causal retrieval.

The selections expose a concrete failure mode. For HotpotQA, evidence lies at token spans 107–137 and 543–588, while routed chunks lie near 1216–1318. For QASPER, evidence lies at 0–317, while selected chunks lie near 4032–4192. Post-RoPE mean-key gists and an unadapted top-layer query systematically prefer irrelevant late-source chunks in these two probes.

10 Routing Representation Study

The smoke failure motivates a narrower test: is the post-RoPE representation appropriate for semantic routing? For token j at position p_j, the baseline pools

$$g_{\text{post}} = \frac{1}{m} \sum_{j=1}^{m} R(p_j) K_{\text{pre},j}, \tag{5}$$

which mixes content with different rotary phases. The position-neutral alternative pools $g_{\text{pre}} = m^{-1} \sum_j K_{\text{pre},j}$. A third representation pools the attention-input hidden states. These are hypotheses about ranking only; selected detail memory remains Qwen's post-RoPE K and native V in every condition.

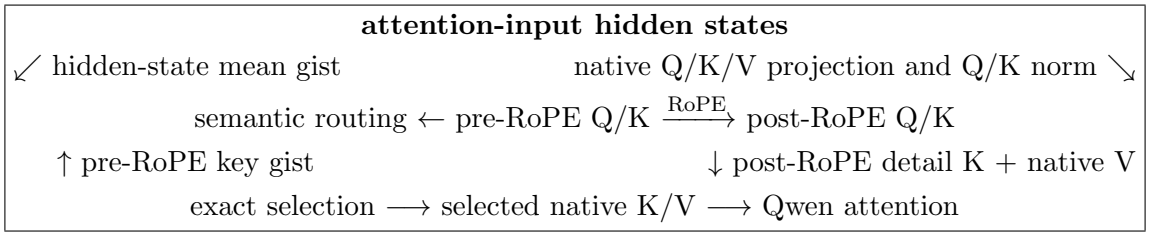

Figure 2: Routing and transport are separate. Pre-RoPE or hidden-state features rank chunks; the attention path always materializes post-RoPE native K/V.

The code makes this independence executable. Routing returns stable *selected chunk identities*; a separate core entry point applies the common budget and materializes their payload. A fixed-selection test supplies the same identity set with ordinary-router and oracle metadata. It obtains bit-identical post-RoPE K, identical V, and exactly identical native attention output. Router choice can change the set, but it cannot silently change what a fixed set means to attention.

10.1 Protocol

Stage 1 uses eight HotpotQA and eight QASPER validation examples, matched across representations. The frozen checkpoint, last-layer placement, one mean gist per chunk, 128-token encoding blocks, zero routing overlap, and exact tensorized search remain fixed. We compare post-RoPE keys, normalized pre-RoPE keys, and attention-input hidden states with 32-token routing chunks and $k \in \{3, 8, 16\}$. Evidence text is mapped to exact tokenizer spans. The final-token question query is captured at the same layer: GQA query groups are averaged into native K/V width for both key representations, while hidden-state routing uses the matched final hidden vector.

The experiment scores all chunks and does not generate answers. This keeps evidence ranking separate from decoding and from the hard materialization budget. Stage 2 carries the best two representations to 64- and 128-token chunks. Stage 3 confirms the best sparse configuration on two deterministic 32-example subsets with different seeds: 64 evaluations covering 60 unique examples. No model or router parameter is trained.

10.2 Representation and position bias

Table 4 reports Stage 1 means over both datasets. The selected fractions at $k = 3, 8, 16$ are 0.040, 0.107, and 0.213 for every representation. Post-RoPE scores are strongly correlated with later source position. Both position-neutral alternatives remove that correlation and improve evidence ranking.

Table 4: Stage 1 routing representation comparison, 16 matched examples and 32-token chunks.

Routing representation	Recall@3	Recall@8	Recall@16	MRR	Score-position r
Post-RoPE key mean	0.125	0.250	0.313	0.131	0.652
Pre-RoPE key mean	0.313	**0.563**	**0.750**	**0.301**	0.009
Hidden-state mean	**0.438**	0.500	**0.750**	0.276	-0.021

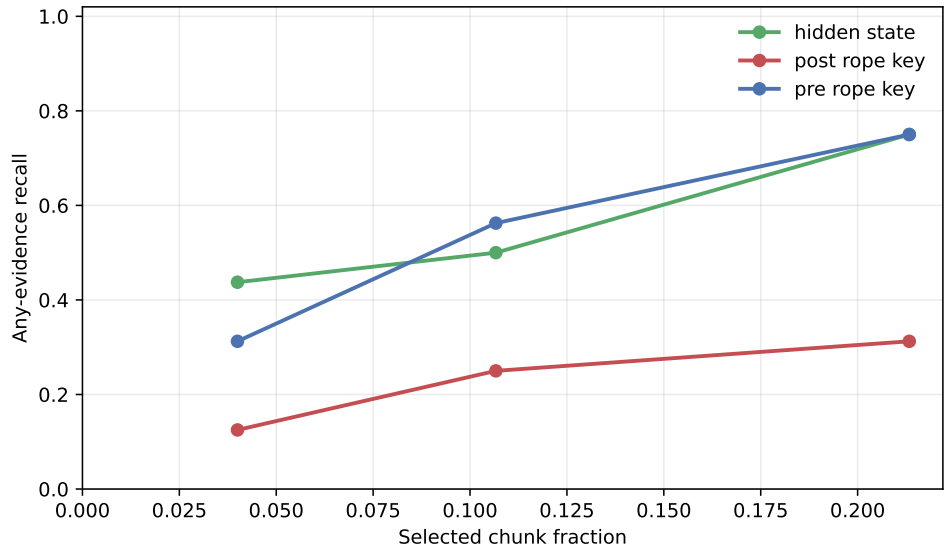

Figure 3: Any-evidence recall against routed chunk fraction in Stage 1. Pre-RoPE and hidden-state routing dominate the post-RoPE baseline, but high recall still requires broader selection.

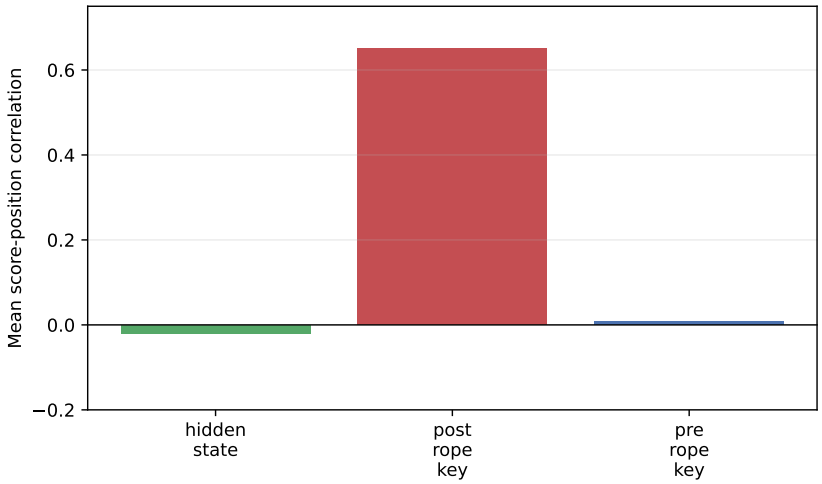

Figure 4: Mean chunk-score correlation with normalized source position. The late-source effect is specific to post-RoPE mean-key routing in this comparison.

The result supports the RoPE-contamination hypothesis: the post-RoPE representation is a poor mean-pooled semantic index. It does not establish that pre-RoPE keys are the best router. Hidden-state means provide the best low-k recall, while pre-RoPE keys provide the best MRR and recall@8. Native key space and semantic retrieval space need not coincide even before rotation.

10.3 Chunk size and breadth

Table 5 shows that larger chunks sometimes raise recall, but they also make a fixed k cover much more of the source. At 128 tokens and $k = 8$, both representations reach 0.688 recall while

selecting 41.3% of chunks. At $k = 16$, the selected fraction is 71.5%. These are useful boundary measurements, not sparse-routing wins.

Table 5: Stage 2 chunk-size sweep over the 16 matched examples. Fraction is reported at $k = 8$.

Representation	Chunk tokens	Recall@3	Recall@8	Recall@16	Selected fraction@8
Pre-RoPE key	32	0.313	0.563	0.750	0.107
Pre-RoPE key	64	0.250	0.438	0.813	0.211
Pre-RoPE key	128	0.375	0.688	0.875	0.413
Hidden state	32	0.438	0.500	0.750	0.107
Hidden state	64	0.313	0.625	0.813	0.211
Hidden state	128	0.438	0.688	0.813	0.413

The expected larger-k recall/cost tradeoff is confirmed. The predicted degradation of post-RoPE pooling with larger chunks is not tested after that representation failed Stage 1; the two finalists do not degrade monotonically. Their larger-chunk gains are inseparable from coarser evidence coverage and a larger selected source fraction.

10.4 Independent confirmation and gate

Hidden-state routing with 32-token chunks is selected for confirmation because it gives the best recall at $k = 3$. Table 6 combines both seeds. Materialization is capped at 128 tokens, so routed breadth continues to grow at $k = 8$ and 16 while the physically materialized fraction stays near 0.059.

Table 6: Hidden-state confirmation over 64 evaluations. Intervals are 95% Wilson intervals.

k	Any recall	95% interval	All recall	Selected fraction	Materialized fraction
3	0.391	[0.281, 0.513]	0.063	0.045	0.045
8	0.578	[0.456, 0.691]	0.203	0.119	0.059
16	0.797	[0.683, 0.877]	0.500	0.238	0.059

At $k = 3$, HotpotQA recall is 0.625 but QASPER recall is 0.156; combined MRR is 0.326. The combined score-position correlation is -0.077, so the original late-position failure is not the remaining blocker. The predeclared promotion gate requires combined recall@3 ≥ 0.70, each dataset ≥ 0.60, MRR ≥ 0.50, selected fraction ≤ 0.10, and absolute position correlation ≤ 0.15. Only the last two conditions pass. Qwen therefore does not advance to Llama.

One hidden-state gist adds 1.57% over detail K/V bytes at 32-token chunks. On the GTX 950M, mean packed-index construction is 3.50 ms, warm exact routing plus selection is 1.90–2.74 ms, and the isolated `torch.topk` primitive is about 0.05 ms. These are single-request Python measurements, not serving-throughput claims.

10.5 Contiguous multi-gist diagnostic

Before introducing learned parameters, we test whether one arithmetic mean simply erases useful local structure. Let the $T = 32$ hidden states in parent chunk c be divided into G balanced, contiguous, nonempty subspans $S_{c,i}$. The routing representation and parent score are

$$g_{c,i} = \frac{1}{|S_{c,i}|} \sum_{t \in S_{c,i}} h_t, \qquad s_c = \max_{i \leq G} \cos(q, g_{c,i}). \tag{6}$$

For $G = 1, 2, 4, 8$, nominal subspan widths are 32, 16, 8, and 4 tokens. Top-k selection remains over parent chunks, not gists. A winning subspan therefore identifies one parent, whose post-RoPE native K/V is materialized once under the same 128-token budget. This isolates routing-index resolution from K/V storage and materialization granularity.

The implementation was fixed before the run at repository commit `0205df8`. Balanced pooling and local-span metadata occupy `gists/segment_mean.py`, lines 11–56. Reference publication calls the common gist interface at `hf/injection.py`, lines 163–190. The existing packed router stores $[C, G, D]$ gist tensors, masks short final chunks, reduces with a maximum, and retains the winning gist index at `memory.py`, lines 487–593. A unit gate requires $G = 1$ to be bit-exact with the historical single mean.

The exploratory sweep uses the same 16 matched examples as Stage 1. Since every multi-gist condition loses at recall@3, no intervention qualifies as a candidate under the predeclared selection rule. We nevertheless run the three interventions on both established 32-example subsets to test whether the negative direction persists. Table 7 combines 64 evaluations per G; four examples occur in both subsets, leaving 60 unique examples.

Table 7: Contiguous hidden-state means with fixed 32-token parents. Overhead is routing-gist bytes divided by post-RoPE detail K/V bytes.

G	Tokens/gist	Recall@3	95% interval	MRR	Recall@8/16	Overhead
1	32	**0.391**	[0.281, 0.513]	**0.326**	0.578/0.797	1.57%
2	16	0.313	[0.212, 0.434]	0.292	0.516/0.734	3.14%
4	8	0.266	[0.173, 0.385]	0.234	0.500/0.688	6.28%
8	4	0.281	[0.186, 0.401]	0.249	0.563/**0.828**	12.56%

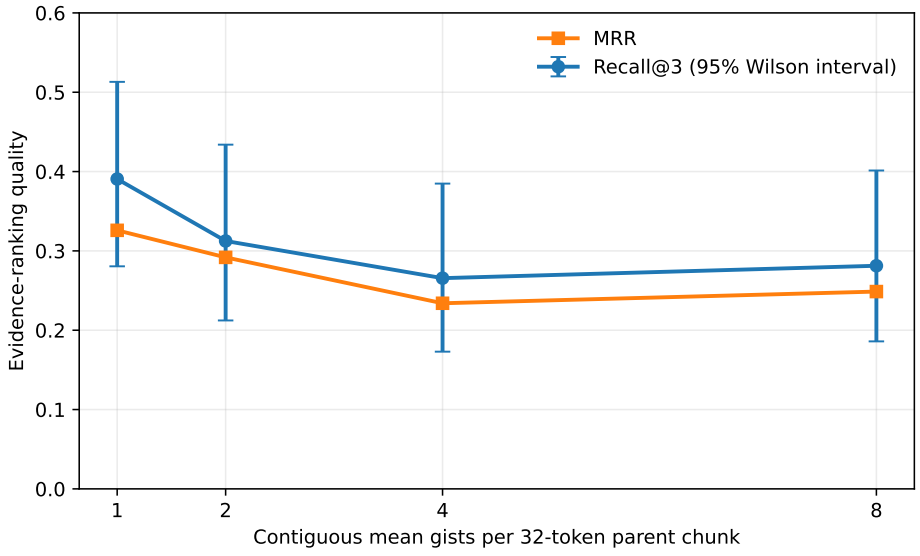

Figure 5: Sparse evidence ranking does not improve as a parent chunk receives more contiguous mean gists. Intervals are Wilson intervals over 64 evaluations.

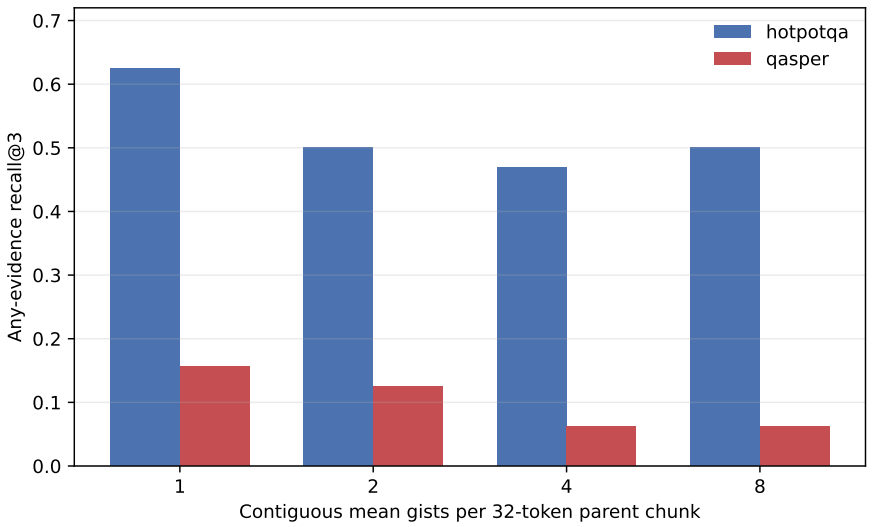

Figure 6: Recall@3 declines on both datasets. The QASPER weakness is not repaired by finer parameter-free means.

At $k = 3$, paired recall changes relative to $G = 1$ are 4 gains versus 9 losses for $G = 2$, 5 versus 13 for $G = 4$, and 5 versus 12 for $G = 8$; exact two-sided McNemar p-values are 0.267, 0.096, and 0.143. The intervals overlap and the paired tests do not cross 0.05, so this is not evidence that all multi-gist schemes are harmful. It is evidence against the specific proposed remedy: untrained contiguous means with max aggregation do not improve sparse ranking here. The $G = 8$ recall@16 increase is a broad-retrieval effect, not a sparse win; recall@3 and MRR remain below the single-mean baseline.

Routing-index bytes scale nearly linearly with G, while selected/materialized parent fractions and active K/V bytes remain identical at fixed k. Packed-index construction is noisy (3.50, 11.60, 4.62, and 4.81 ms for $G = 1, 2, 4, 8$), and warm routing remains 1.93–2.79 ms at this small scale; neither timing sequence supports a throughput claim. The result rejects H5, leaves the proposed H6 saturation inapplicable, and supports H7 for bytes and K/V materialization. It makes an evidence-supervised frozen-Qwen router the next controlled intervention before broader clustering, query-conditioned gists, or another model family.

10.6 Token-resolution hidden-state diagnostic

One remaining parameter-free question is whether useful evidence survives in individual hidden states even when every tested mean loses it. With the parent width fixed at 32 tokens, we set $G = 32$ in the same balanced segment implementation. Each nonempty segment is then one token, so the parent score becomes

$$s_c = \max_{t \in c} \cos(h_{\text{query}}, h_t). \tag{7}$$

This is an upper-resolution diagnostic, not a proposed production index. It preserves parent identity, selected fraction, post-RoPE K/V, and the materialization budget, while increasing the number of routing vectors by about 32 times.

Table 8: Matched 16-example hidden-state diagnostic with fixed 32-token parents.

Router	Recall@3	Recall@8	Recall@16	MRR	Index/detail-KV %
One parent mean ($G = 1$)	**0.438**	0.500	0.750	0.275	1.57
Maximum over tokens ($G = 32$)	0.375	**0.625**	**0.813**	**0.314**	50.00

The aggregate MRR gain is not consistent across datasets. HotpotQA improves from 0.338 to 0.469, with unchanged recall@3 of 0.625; QASPER MRR falls from 0.213 to 0.160 and recall@3 falls from 0.250 to 0.125. Figure 7 shows the corresponding breadth tradeoff. The result does not support token-max as a general sparse-routing remedy. It does suggest that pooling loss can matter for some HotpotQA ranks, while QASPER's dominant failure is weak query/evidence alignment even at token resolution.

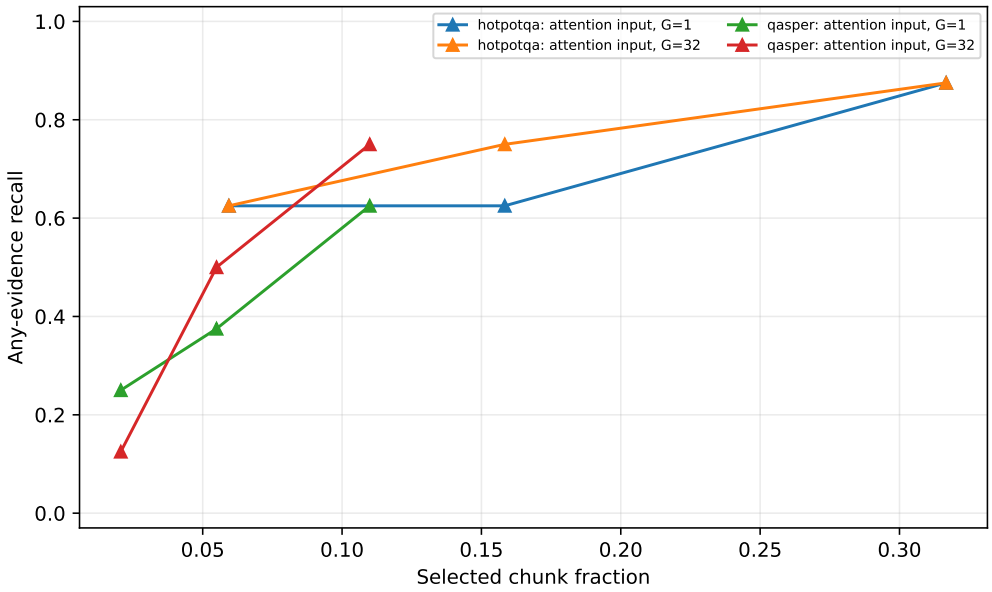

Figure 7: Token-resolution maxima improve some broader ranks but reduce sparse QASPER recall. Selected parent fractions, not routing-vector fractions, are shown.

The token-max index contains a mean 3,255 candidate gists per example versus 102 for one mean. Its measured index-construction time is about 40.1 ms versus 16.4 ms on this small run, while warm top-k timing is too noisy to interpret. The decisive cost is persistent routing state: 50% of detail-K/V bytes is incompatible with the intended compact-index role.

10.7 Center-Position RoPE Gists

The final zero-parameter control asks whether post-RoPE routing failed because it averaged keys expressed in different positional frames. For a contiguous span S_c with known source positions

$p_s, \ldots, p_e$, we distinguish

$$g_{\text{pre}} = \frac{1}{|S_c|} \sum_{t \in S_c} K_{\text{pre},t}, \tag{8}$$

$$g_{\text{postmean}} = \frac{1}{|S_c|} \sum_{t \in S_c} R(p_t) K_{\text{pre},t}, \tag{9}$$

$$g_{\text{center}} = R(p_c) g_{\text{pre}}, \qquad p_c = \frac{p_s + p_e}{2}. \tag{10}$$

The centered form pools in one position-neutral frame and introduces a single representative position afterward. Its matched routing query is $Q_{\text{post}} = R(q)Q_{\text{pre}}$. Even-length spans use their exact half-integer center through Qwen's installed rotary module; no independent RoPE approximation is introduced. This changes routing gists only. Every selected parent still materializes the same native post-RoPE K and native V.

The implementation is revision-pinned at commit `4c2a0f0`. Configuration and policy validation are in `hf/config.py`, lines 11–75. Qwen reshapes each pooled $[G, H_{kv} d_h]$ gist to native heads, asks the installed rotary module for float-position cosines and sines, applies the family helper, and restores packed routing layout at `hf/qwen.py`, lines 80–105. Span centers and persistent source-span metadata are constructed independently of packed-index order at `hf/injection.py`, lines 105–148; publication invokes that transform only after ordinary pre-RoPE pooling at lines 207–229.

Table 9 reports the same 16 examples, 32-token parents, and $k \in \{3, 8, 16\}$ used in the representation study. Centered-RoPE is directionally better than the post-RoPE mean, with one paired Recall@3 gain and no losses, but the exact two-sided McNemar p-value is 1.0 because only one example changes. Its position correlation falls from 0.651 to 0.567 but remains far above the position-neutral alternatives.

Table 9: Matched centered-RoPE representation control with one gist per 32-token parent.

Router	Recall@3	Recall@8	Recall@16	MRR	Position r
Post-RoPE mean	0.125	0.250	0.313	0.131	0.651
Centered-RoPE mean	0.188	0.313	0.438	0.168	0.567
Pre-RoPE mean	0.313	**0.563**	**0.750**	**0.301**	0.008
Hidden-state mean	**0.438**	0.500	**0.750**	0.275	-0.021

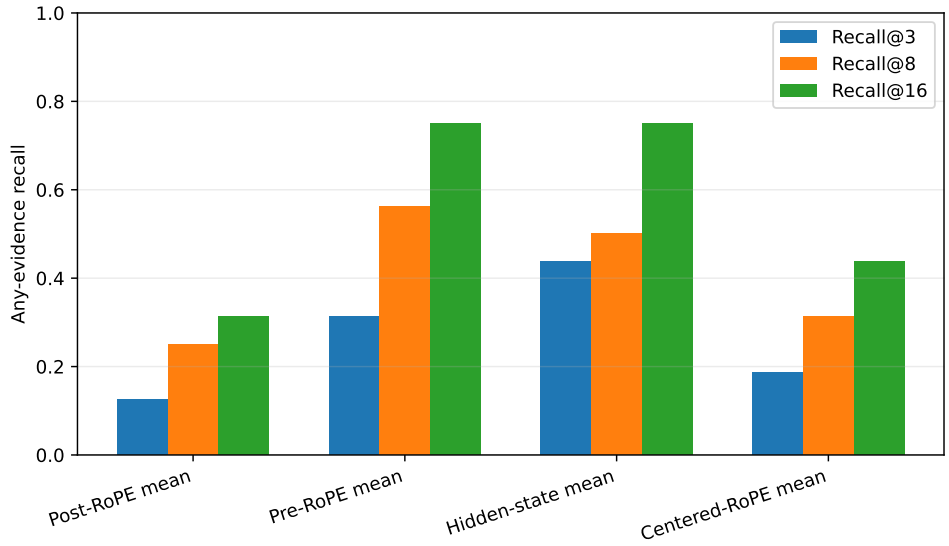

Figure 8: Centered pooling repairs part of the post-RoPE mean failure, but position-neutral pre-RoPE and hidden-state routing retain substantially better sparse evidence recall.

For segment means, each subspan receives its own exact center. At $G = 1, 2, 4, 8$, nominal spans contain 32, 16, 8, and 4 tokens, with maximum center-approximation errors of 15.5, 7.5, 3.5, and 1.5 positions. Table 10 compares evidence ranking with Spearman correlation against the chunk ordering produced by maximum native token-QK score.

Table 10: Centered segment gists. QK rank is Spearman correlation with native token-QK maximum ordering; overhead is routing-index bytes divided by detail-K/V bytes.

G	Recall@3	MRR	Position r	Native-max QK rank	Overhead
1	0.188	0.168	0.567	0.521	1.57%
2	0.125	0.134	0.588	0.602	3.14%
4	**0.250**	0.139	0.557	0.710	6.28%
8	**0.250**	**0.180**	0.455	**0.874**	12.56%

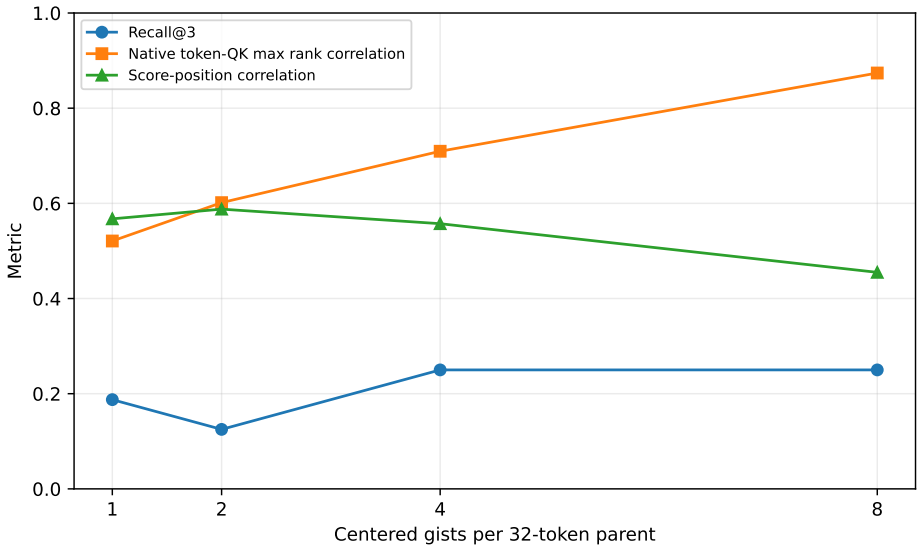

Figure 9: Finer centered gists increasingly reproduce native token-QK maximum ordering, but semantic Recall@3 remains weak. Native attention compatibility and evidence relevance are different objectives.

The mechanistic hypothesis is therefore supported, but the routing-quality hypothesis is not. At $G = 8$, native-maximum rank correlation reaches 0.874 and position bias declines, yet Recall@3 is 0.250 and QASPER Recall@3 remains zero. Exact, floor, and ceil center policies all produce Recall@3 of 0.125 on the matched eight-example fractional control; their score changes are below 0.0074, so fractional-position handling is not the blocker. E1 is directionally supported, E2 is rejected, E3 is supported, and E4 receives only weak partial support. This closes the current zero-parameter routing transformations and advances the frozen-Qwen learned hidden-state router.

10.8 Representing the Information Need

The preceding experiments vary the memory representation while using the final attention-input state as the retrieval query. We next hold memory fixed at one hidden-state mean per 32-token chunk and ask whether the information need should aggregate several prompt states. For prompt states $h_1, \ldots, h_t$, the recent exponential candidate is

$$r_q = \frac{\sum_{i=0}^{W-1} 2^{-i/H} h_{t-i}}{\sum_{i=0}^{W-1} 2^{-i/H}}, \tag{11}$$

where H is a token half-life. Exact-question variants apply the same weighting only to states whose tokenizer offsets overlap the known benchmark question. Question boundaries are evaluation metadata, not an assumption about ordinary generation.

The broad validation stage compares last state; uniform windows 4, 8, and 16; exponential $W = 16$ with $H = 2, 4, 8$; a linear control; question mean; and question decay $H = 4, 8$ on 8 examples per dataset. Last-state recall@3 is 0.438. Question decay at $H = 4$ is the strongest aggregate at 0.375 and raises HotpotQA MRR from 0.338 to 0.526, but lowers QASPER recall@3 from 0.250 to 0.125. Refinement adds half-lives 2 and 16 and windows 2 and 32. We carry the best question-decay and long-uniform tradeoffs to 32 identity-disjoint confirmation examples.

Table 11: Held-out query-representation confirmation with fixed hidden-state memory gists.

Query	Recall@3	Recall@8	Recall@16	MRR	Position r
Last state	**0.469**	**0.719**	**0.844**	**0.413**	-0.075
Question decay, $H = 2$	0.250	0.688	0.813	0.279	-0.014
Uniform suffix, $W = 32$	0.313	0.469	0.813	0.348	-0.076

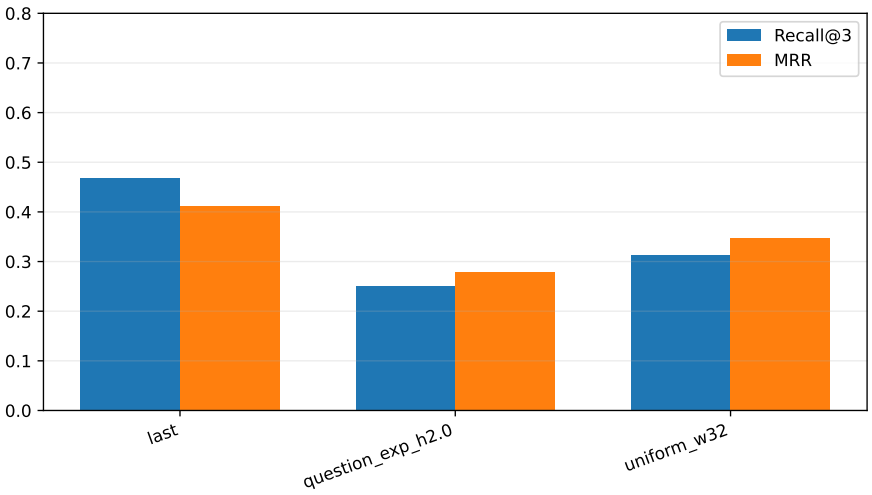

Figure 10: Neither selected aggregate reproduces the last-state sparse ranking on held-out questions. Aggregation changes query geometry substantially but does not improve relevance.

The aggregate vectors are not trivial perturbations: their mean cosine with the last state is 0.459 for question decay and 0.576 for the 32-token mean. Nonetheless, both lower sparse recall and MRR. A post-hoc length split suggests that the long uniform window is less harmful on long questions, but the bins contain only 21 and 11 examples and confound dataset composition. We do not promote a length-conditioned policy. The simplest last-state query remains canonical.

10.9 Learning Relevance Geometry on Frozen Hidden States

The negative resolution and query-pooling results motivate a learned metric rather than another hand-designed representation. We precompute frozen last-state queries and mean chunk gists for 48 train, 16 validation, and 32 test examples, split by QA identity. The sets contain 4,707, 1,634, and 3,084 candidate chunks, with no identity overlap. All 596,049,920 Qwen parameters have `requires_grad=False`. For positive chunks P_q among all in-document candidates C_q, the first adapters minimize

$$\mathcal{L}_q = -\log \frac{\sum_{c \in P_q} \exp(s(q,c)/\tau)}{\sum_{c \in C_q} \exp(s(q,c)/\tau)}. \tag{12}$$

The objective extension also evaluates pairwise margin ranking,

$$\mathcal{L}_q^{\text{margin}} = \frac{1}{|P_q||N_q|} \sum_{p \in P_q} \sum_{n \in N_q} \max(0, m - s(q,p) + s(q,n)), \tag{13}$$

with $m = 0.2$. Exhaustive candidates include current-router false positives, lexical competitors, and position-matched negatives. Separate controls sample seven mixed negatives per positive and then refresh that pool once with top false positives from the learned router.

We compare shared linear, asymmetric linear, and asymmetric two-layer MLP projections at widths 64 and 128 with five seeds, then add width 256 and the margin objective without opening the test set for selection. Validation recall@3 followed by MRR selects asymmetric linear width 128 with the margin objective (validation recall@3/MRR 0.838/0.692). It contains 262,144 parameters, 1.0 MiB in float32, or 0.044% of Qwen. Table 12 reports the held-out result.

Table 12: Five-seed held-out learned-routing controls. The confidence interval is the 95% half-width across adapter seeds. Selection uses validation only.

Metric	Recall@3	Recall@8	MRR	Hotpot R@3	QASPER R@3
Cosine, last state	0.469	0.719	0.413	0.813	0.125
Asymmetric-128, contrastive	0.563 ± 0.039	0.806	0.516	0.400	0.725
Shared-256, contrastive	0.631 ± 0.058	0.819	0.558	0.500	0.763
Asymmetric-128, margin (selected)	$\mathbf{0.631 \pm 0.069}$	0.806	0.498	0.588	0.675
Asymmetric-128, mixed negatives	0.538 ± 0.069	0.819	0.494	0.313	0.763
Asymmetric-128, mined round 1	0.506 ± 0.042	0.825	0.479	0.288	0.725

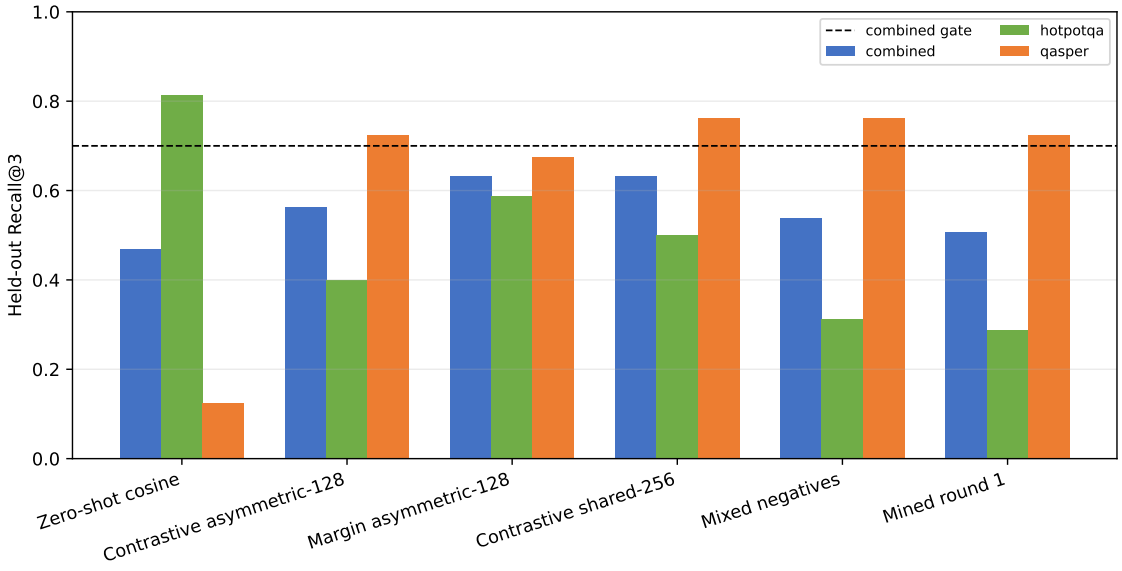

Figure 11: The margin objective balances HotpotQA and QASPER better than the contrastive adapter. Larger width and harder sampled negatives do not cross the predeclared combined gate.

The selected adapter improves recall@3 over zero-shot cosine by 0.163 ± 0.069 across the five paired seeds. It passes recall@8 (0.806), both per-dataset recall@3 thresholds, position bias ($r = -0.026$), and width gates, but fails combined recall@3 (0.631 versus 0.70). Shuffled-label recall@3 is 0.125. Hotpot-to-QASPER and QASPER-to-Hotpot transfer both reach only 0.213. The adapter learns evidence alignment rather than a label-independent shortcut, but the geometry is not yet domain-general.

The earlier 2×2 contrastive ablation also constrains the query interpretation. With cosine scoring, last/question-decay recall@3 is 0.469/0.250; with the learned metric it is 0.563/0.519.

Learning removes much of the aggregate query's deficit, but aggregation adds no independent gain. The last state therefore remains the canonical query.

Increasing representation resolution is less effective than learning alignment. The selected single-mean adapter reaches recall@3 0.631 with a routing index equal to 0.392% of native detail K/V bytes. The earlier zero-shot token-resolution maximum reaches 0.375 while indexing 50%; it uses a smaller cohort and is therefore descriptive rather than paired. Width 256 doubles index cost to 0.784% without validation selection. Figure 12 shows the matched learned conditions. The raw 1024-dimensional float32 hidden-state index occupies 3.14% of detail K/V; the selected 128-dimensional index is eight times smaller.

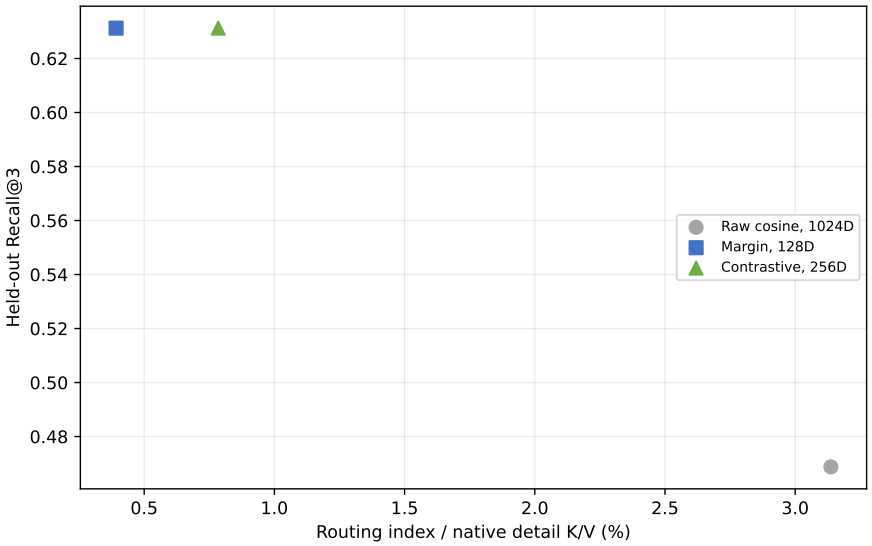

Figure 12: Held-out sparse recall versus routing-index storage for matched frozen-feature conditions. Width 128 improves the quality/cost point; width 256 adds cost without selection.

The integration changes routing only. At revision `a657bc7`, query pooling is implemented in `hf/query.py`, lines 77–117. Shared/asymmetric projections and cosine normalization are in `hf/routing_adapter.py`, lines 10–74. Reference publication projects only computed gists in `hf/injection.py`, lines 238–246, while the live query is projected in `hf/qwen.py`, lines 188–202. Native detail K/V bypass both projections. An invariant test confirms exact K/V equality with and without the adapter.

The implementation keeps this experiment auditable. At revision `604926e`, frozen feature publication sets every Qwen parameter non-trainable in `precompute_router_features.py`, lines 117–126. Deterministic mixed-negative selection is in `train_learned_router.py`, lines 171–239; contrastive and margin losses are at lines 246–283. These training paths consume stored vectors and cannot update native K/V.

Finally, the validation-selected seed is exercised through cache lookup, materialization, and generation on four held-out examples per dataset. It recalls evidence in 5/8 cases, covers 47.9% of annotated targets, stores a routing index equal to 0.393% of detail-K/V bytes, and incurs no native-limit violation. Frozen-feature adapter scoring takes 2.05 ms/example and full ranking 2.67 ms on the GTX 950M, versus 2.18 ms for raw cosine; these small synchronized timings are descriptive rather than a serving benchmark. Answer F1 is 0.090 with PRA disabled and 0.090 with learned routing enabled. Better selection does not establish causal use; further router capacity is not a

justified response to that separate failure.

10.10 Canonical end-to-end confirmation

After establishing the routing/payload boundary, we run one HotpotQA example, one QASPER example, and one synthetic displaced prompt through the explicit default: attention-input hidden-state routing followed by native post-RoPE K/V materialization. Table 13 reports prefill routing and answer generation separately.

Table 13: Compact end-to-end confirmation. All conditions materialize three 32-token parents.

Source	Recall	Selected	K/V tokens	F1	Violations
HotpotQA explicit URI	0.000	0.071	96	0.000	0
QASPER explicit URI	0.000	0.023	96	0.000	0
Synthetic `#__head`	1.000	0.375	96	0.000	0

The explicit cases reproduce the earlier causal-negative result: neither selected set overlaps annotated evidence and neither answer is correct. The 416-token implicit-prompt case displaces 256 tokens, keeps a 160-token direct tail, selects the chunk containing the early "cobalt" fact, and combines 96 memory tokens with the tail at the 256-token native bound. It nevertheless does not generate the target answer. This is a useful separation of claims: native transport, logical positions, and even evidence selection can work in one probe without establishing that a single frozen upper layer uses the memory causally for decoding.

11 Systems Observations

The first run reaches 1,345,618,432 allocated GPU bytes, including the complete FP16 model and temporary activations. Full reference K/V remains on CPU; only selected detail K/V moves to the GPU. This is evidence that the residency path executes, not a memory-savings comparison: no matched dense long-context peak is measured here. Likewise, cold source construction, warm routing, generation, and transfer are reported separately so cache reuse is not hidden.

The artifacts keep three quantities distinct: compact routing-index bytes, resident detail-K/V bytes, and selected/materialized K/V bytes. The token-max experiment changes the first by roughly 32 times while holding the latter two fixed at a given parent selection and budget.

The current HF path is intentionally eager and unfused. Variable memory rows are padded into a rectangle before Qwen's native eager function. Flash attention, SDPA, custom kernels, ANN routing, asynchronous overlap, and serving concurrency remain outside this correctness phase.

12 Limitations and Next Experiments

The study has one family, one 596M-parameter checkpoint, and one PRA layer placement. The representation/chunk and exploratory multi-gist sweeps use 16 matched examples; confirmation uses two deterministic subsets rather than a full validation corpus, and four examples overlap between seeds. The main sweeps measure annotated-span ranking; the three-probe end-to-end check also reports EM/F1 but is not an accuracy estimate. A model cannot receive causal credit for an answer when selected K/V excludes the evidence. There is no matched learned text retriever, multi-layer placement study, LoRA adaptation, serving-throughput study, or full dense-context quality comparison.

The tiny learned adapter establishes that evidence supervision can reshape frozen hidden states, especially on QASPER, but validation/test instability and weak cross-domain transfer prevent a general routing claim. The next experiment should increase split size and label diversity before adding adapter capacity. If retrieval becomes stable while generation remains unchanged, the next mechanism belongs to memory-use alignment rather than routing. The contiguous-mean, token-max, centered-RoPE, and query-pooling results close the current simple zero-parameter sweeps. Llama remains blocked until Qwen passes the declared gate. Gemma remains later.

13 Related Work

PRA differs from text retrieval-augmented generation, which retrieves text and lets the model encode it in the current prompt [5]. It retrieves layer-native K/V after a separate bounded encoding path. This makes PRA complementary to RAG: a text retriever may choose documents before PRA routes chunks inside cached model state. Sparse-attention systems such as Longformer and BigBird constrain token-token connectivity inside a forward pass [2, 11]; PRA instead materializes selected state from an external addressable cache. KV-cache systems emphasize serving capacity and token retention policies [4, 12, 6]. Paper 2 asks whether selection can be inserted beneath an existing pretrained family without changing its disabled behavior.

14 Conclusion

One shared PRA core now serves both the controlled model and a thin Qwen adapter. The frozen Qwen3-0.6B checkpoint passes bit-exact disabled parity, preserves native RoPE and GQA cache layout, replays selected native K/V through generation, and executes an implicit long-prompt head under a hard native-operation bound. The routing study adds a second architectural result: semantic routing state and native attention state are separate interfaces. Attention-input hidden-state gists form the canonical compact index, while a fixed selected identity always materializes the same native post-RoPE K and native V. A direct invariant test establishes identical K/V and attention output for router and oracle labels that provide the same set.

That correction is necessary but insufficient. Hidden-state routing reproduces useful evidence ranking above the post-RoPE baseline, yet sparse recall and MRR remain below the declared gate, especially on QASPER. Selecting nearly one quarter of chunks raises recall@16 to 0.797, but does not constitute a sparse-routing solution. Contiguous multi-gist means increase index size and occasionally improve broad recall without improving recall@3 or MRR. Token-resolution maxima raise some HotpotQA ranks but worsen sparse QASPER recall at 50% routing-index overhead. Centered-RoPE gists repair part of incompatible-frame averaging and, with finer segments, closely reproduce native token-QK ordering. They still trail pre-RoPE and hidden-state routing on annotated evidence. This directly supports the routing/payload split: native attention compatibility is not semantic evidence relevance. Recent-state and question-span query aggregation also fail to beat the last state. A tiny margin-trained metric improves combined recall@3 by 0.163 and balances HotpotQA/QASPER without positional bias, showing that relevance geometry is a real part of the bottleneck. More dimensions and harder sampled negatives do not improve it. The validation-selected adapter still misses the held-out gate and transfers poorly. End-to-end answer quality remains unchanged even when evidence is selected. The next work must separate data-limited routing generalization from memory-use alignment, not claim solved long-context retrieval or proceed to Llama.

A Code-Level Adapter Reimplementation Guide

This appendix gives the minimum code structure needed to reproduce the integration without copying the Qwen implementation. Source references use physical line numbers at repository commit `db50846`. This checkpoint makes attention-input hidden-state routing the explicit default and exposes the fixed-selection materialization boundary. The line references are deliberately revision-pinned because upstream Transformers APIs and local diagnostics will move over time.

A.1 Tensor and ownership contract

A new family should add projection, position, cache, mask, kernel, and output conventions in its adapter, while leaving selection and memory policy in the shared core.

The canonical tensor contract is

$$Q \in \mathbb{R}^{B \times H_q \times T_q \times d_h}, \qquad K, V \in \mathbb{R}^{B \times H_{kv} \times T_k \times d_h}. \tag{14}$$

Reference entries use $B = 1$ and keep H_{kv}, including for GQA or MQA. They are not expanded to H_q in persistent storage. The controlled replay implementation shows the only required expansion point in `memory_batching.py`, lines 216–290: after memory and local K/V are joined, native K/V heads may be repeated for the attention computation. Qwen's installed eager kernel performs the corresponding operation in the production adapter.

A.2 Minimal forward-path skeleton

The following pseudocode is the adapter's essential control flow. It is intentionally family-neutral; "native" operations must call the installed model implementation rather than reimplementing its mathematics.

```
forward(hidden, native_positions, native_mask, past):
    if capture_is_armed:
        q_pre, k_pre, v = native_project_and_norm(hidden)
        q_post, k_post = native_position(q_pre, k_pre)
        save_transient_routing_features(hidden, q_pre, q_post, k_pre)
        save_post_position_detail_kv(k_post, v)

    if memory_is_disabled or reference_cache_is_empty:
        return original_attention(hidden, native_positions,
                                  native_mask, past)

    q_pre, k_pre, v = native_project_and_norm(hidden)
    q_post, k_post = native_position(q_pre, k_pre)
    if past is not None:
        k_post, v = past.update(k_post, v, layer_id, native_kwargs)
    route_q = matched_query(hidden[:, -1, :], q_pre, q_post)
    memory = shared_core.prepare_memory(q_post, route_q,
                                        direct_tokens=k_post.shape[2])
    if memory.is_empty:
        return native_attention_and_output(q_post, k_post, v, native_mask)
    k, v, mask = shared_core.combine_local_and_memory_kv(
```

```
22          k_post, v, memory, native_mask, query_tokens=q_post.shape[2])
23      return native_attention_and_output(q_post, k, v, mask)
```

Lines 8–10 are a correctness boundary, not an optimization. The Qwen implementation delegates to the untouched module at `hf/qwen.py`, lines 190–201, which makes disabled behavior auditable and bit-exact. In the active path, projection and RoPE occur at lines 204–207 and `Cache.update` occurs once at lines 208–215. Lines 216–221 choose the matched routing query without altering the post-RoPE attention query. If routing returns no materialized memory, lines 224–234 call the native kernel directly. Delegating at that point would execute a second cache update and duplicate generation history.

Memory is composed only after the native cache update. The shared operation at `core.py`, lines 394–436, pads variable memory rows, prepends memory K/V, creates an all-visible additive mask for valid memory positions, masks padding, and concatenates the unchanged local mask. The adapter then invokes Qwen's eager function and one pretrained `o_proj` at `hf/qwen.py`, lines 92–110 and 246–258. Thus local and memory positions share one softmax and one output projection.

A.3 Optional learned routing metric

The learned adapter is a replaceable routing component, not part of native attention. For an asymmetric linear metric,

$$r_q = \mathrm{norm}(W_q h_q), \qquad r_c = \mathrm{norm}(W_c g_c), \qquad s(q,c) = r_q^\top r_c. \tag{15}$$

A shared adapter aliases $W_q = W_c$. An independent implementation needs only the following boundary, revision-pinned at commit `a657bc7`:

```
01  # Reference publication, after ordinary hidden-state gist pooling
02  routing_gist = adapter.project_memory(routing_gist)
03  cache.store(routing_gist, native_post_rope_k, native_v)
04
05  # Live routing; native attention continues to use q_post
06  information_need = aggregate_query_states(h_in, strategy)
07  routing_query = adapter.project_query(information_need)
08  selected_ids = cache.search(routing_query)
09  k_mem, v_mem = cache.materialize_native_kv(selected_ids)
10  native_attention(q_post, concat(k_mem, k_local),
11                   concat(v_mem, v_local))
```

The projection implementation is `hf/routing_adapter.py`, lines 10–74; checkpoint loading and freezing are lines 77–94. Memory projection occurs at `hf/injection.py`, lines 238–246, and live query projection at `hf/qwen.py`, lines 188–202. Cast hidden features to the adapter parameter dtype at this boundary; the checkpoint is FP32 while Qwen emits FP16 states. Do not cast, project, or recompute native detail K/V. Their exact-equality invariant is the test that prevents a routing retrofit from silently becoming an attention rewrite.

A.4 Publishing a reference

Reference construction is a bounded prefill, not a second attention mechanism. A portable implementation follows this recipe:

```
disable_PRA_memory_during_reference_encoding()
for [block_start:block_end] in bounded_encoding_blocks(source):
    position_ids = logical_source_positions(block_start, block_end)
    arm_post_position_kv_capture(position_ids)
    model(block_ids, position_ids=position_ids, use_cache=False)
    captured = consume_native_kv_for_each_PRA_layer()
    for routing_window in overlapping_windows(captured):
        gist = pool(routing_window.attention_input_hidden_state)
        publish(uri, logical_offsets, post_rope_k, native_v, gist)
        discard_token_level_hidden_state()
restore_previous_memory_state()
```

The concrete loop is `hf/injection.py`, lines 105–210. Encoding block size limits one native model invocation; routing chunk size and overlap independently determine index granularity (lines 128–147). Each chunk slices the captured post-RoPE K/V, chooses transient routing features through `_routing_points` (lines 80–103), computes one or more compact gists, and records logical source offsets (lines 148–203). Keeping those two scales separate is important: changing index granularity must not silently change how the pretrained model encoded the source.

The contiguous multi-gist extension keeps that ownership boundary explicit. For a requested count G, split the routing feature rows into balanced source-order ranges, mean each range, score all resulting $[G, D]$ rows, reduce to one parent score, and preserve the winning local index for diagnostics. Top-k must then operate on parent identities before budgeting; otherwise several winning gists could duplicate one parent K/V payload. The reference implementation is revision-pinned at `0205df8`: pooling is in `gists/segment_mean.py`, lines 11–56, publication remains `hf/injection.py`, lines 163–190, and packed scoring plus max reduction remains `memory.py`, lines 487–593.

The same operation implements displaced long-prompt history. At `hf/injection.py`, lines 212–226, the prefix is published under the reserved `#__head` URI and the direct tail retains continued logical position IDs. Every reference-encoding call and every direct call must satisfy the model-safe native bound even when the logical source is longer.

A.5 Porting another decoder family

A practical port begins by locating seven hooks in the installed family: the attention module inside each decoder layer; Q/K/V projections; any Q/K normalization; positional encoding; generation-cache update and its keyword arguments; native mask convention; and the eager attention plus output projection. Implement the six abstract methods declared at `hf/adapter_base.py`, lines 85–106, then add a narrow type-selection branch analogous to `inject_pra` at `hf/injection.py`, lines 228–253. No checkpoint conversion or parameter copy should be needed.

The recommended validation order is:

1. With memory disabled, require exact logits, hidden states, greedy tokens, and generation- cache lengths against an unwrapped copy (test lines 55–72).

2. Capture a short reference and verify post-position state, logical offsets, device residency, and H_{kv} rather than H_q storage (lines 74–93).

3. Verify matched pre/post capture, representation switching, unchanged post-RoPE detail K/V, GQA grouping, serialization, and device parity (lines 109–214).

4. Supply one selected identity through router and oracle traces; require identical native K/V and attention output (lines 241–280).

5. Compare GQA replay with explicit head repetition on a synthetic tensor (lines 216–239).

6. Exercise an over-limit logical prompt and verify bounded native operations plus continued position IDs (lines 282–313).

7. Generate with active memory and assert that direct cache length grows by exactly one per decoded token (lines 315–330).

Four mistakes deserve explicit tests. Applying position encoding again to stored post-position keys changes their basis. Permanently expanding GQA K/V multiplies memory without adding information. Replacing the family mask with a generic causal mask can alter padding or sliding-window behavior. Finally, calling the wrapped module after manually updating its cache appends the same local token twice. Passing transport and parity tests establishes a faithful adapter; it does not establish that the router selects causally useful evidence, which requires the separate quality controls reported in the main paper.

A.6 Revision-pinned source map

Table 14 connects the preceding control flow to the implementation.

Table 14: Source map for an independent adapter implementation.

Responsibility	File and physical lines
Abstract family boundary	`src/pra_torch/hf/adapter_base.py`, 16–106
Qwen projections, RoPE, native kernel, and output	`src/pra_torch/hf/qwen.py`, 53–110
Matched routing/detail capture	`src/pra_torch/hf/qwen.py`, 109–148; `src/pra_torch/hf/injection.py`, 80–210
Disabled and active runtime paths	`src/pra_torch/hf/qwen.py`, 171–275
Routing query and GQA reduction	`src/pra_torch/core.py`, 92–121
Budget, deduplication, transfer, and materialization	`src/pra_torch/core.py`, 155–392
Fixed-selection identity/payload boundary	`src/pra_torch/core.py`, 335–392
K/V and additive-mask composition	`src/pra_torch/core.py`, 394–436
Selected-layer injection	`src/pra_torch/hf/injection.py`, 234–259
Parity, routing, GQA, head, cache, and selection gates	`tests/test_hf_integration.py`, 55–330

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

[2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[3] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021. Preprint: https://arxiv.org/abs/2105.03011.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: https://arxiv.org/abs/2309.06180.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[6] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: https://arxiv.org/abs/2404.14469.

[7] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[8] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. Preprint: https://arxiv.org/abs/1706.03762.

[10] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018. Preprint: https://arxiv.org/abs/1809.09600.

[11] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.

[12] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: https://openreview.net/forum?id=RkRrPp7GKO.